BERNARD SHAW AI (Misinformation Finder) Using Hybrid Deep Learning and Online Verification ***Project Report***

1. Introduction

Misinformation has become a huge problem today, especially because of how fast information spreads on social media. Many people believe whatever they see online without checking if it is true. This can cause confusion, fear, and even real-world problems.

In this project, I wanted to build a system that can automatically check whether a piece of news is True or False. Instead of depending only on a machine learning model, I combined it with online factchecking so the system becomes more reliable in real-world scenarios. The idea is to use AI for prediction and the internet for verification.

2. Problem Statement

Machine learning models which are trained on an older data often struggle to work efficiently when new types of fake news or misinformation start appearing. At the same time, verifying whether the news given is true or not isn't always easy, which makes the problem even more difficult.

To tackle this, the idea was to build a tool that can:

- Give a quick prediction on whether a piece of news looks fake
- Cross-check the same information using reliable online
- Combine both results to provide a more balanced final answer

By bringing together AI-based prediction and online verification, the system becomes more dependable than relying on just one method alone.

3. Objectives

My main goals for this project were as follows:

1. To train a text classification model using the WELFake dataset
2. To evaluate the performance of the trained model
3. To integrate the system with online APIs such as Wikipedia, GNews, and Google Fact Check for real-time verification
4. To combine the model's output with online verification results to arrive at a final decision
5. To develop a simple Streamlit interface that allows users to enter news and test the system easily

4. Dataset Used

I used the **WELFake Dataset**, which contains over 69,000 news articles. Each sample has a title, text, and a label:

- 1 = Fake
- 0 = Real

I combined the title and text, cleaned them, and then created a train/test split (80:20) for the experiments.

5. Methodology

5.1 Data Preprocessing

I cleaned the text by:

- Removing the URLs
- Lowercasing everything present
- Removing the extra spaces and special characters
- Merging the title and body text

This prepared the dataset for training.

5.2 Model Training

I used **DistilBERT**, a smaller version of BERT, because it is fast and still performs well.
I fine-tuned it for binary classification using:

- Learning rate: 2e-5
- Optimizer: AdamW
- Epochs: 1
- Batch size: 8

The trained model was saved inside models/best_model/.

5.3 Evaluation

I evaluated the model on 14,427 test samples.

The results were:

- Accuracy: 99.87%
- Precision: 95.76%

- Recall: 99.13%
- F1 Score: 98.97%

Confusion Matrix:

	Predicted Real	**Predicted Fake**
Actual Real	7938	70
Actual Fake	57	7299

These results show that the model performs very well, with rare amounts of wrong predictions.

6. Hybrid Online Verification System

The offline model alone is not enough because new events appear every day.
So, I added a second stage where the system checks the news using:

1. **Wikipedia API**
2. **GNews API**
3. **Google Fact Check API**

I designed a simple logic:

- If Google Fact Check marks it false → Final result is FAKE
- If Wikipedia summary strongly matches the topic → TRUE
- If GNews finds supporting articles → TRUE
- If AI says “fake” but online sources don’t confirm → UNCERTAIN
- Otherwise → follow the AI prediction

This hybrid approach improves the reliability of the system when dealing with new or trending topics.

7. System Architecture

The system follows a simple and structured workflow:

1. The user inputs some news
2. Offline AI model analyzes the input and predicts whether it is real or fake
3. Online APIs are then queried to gather supporting information
4. Hybrid logic combines everything to produce the final result

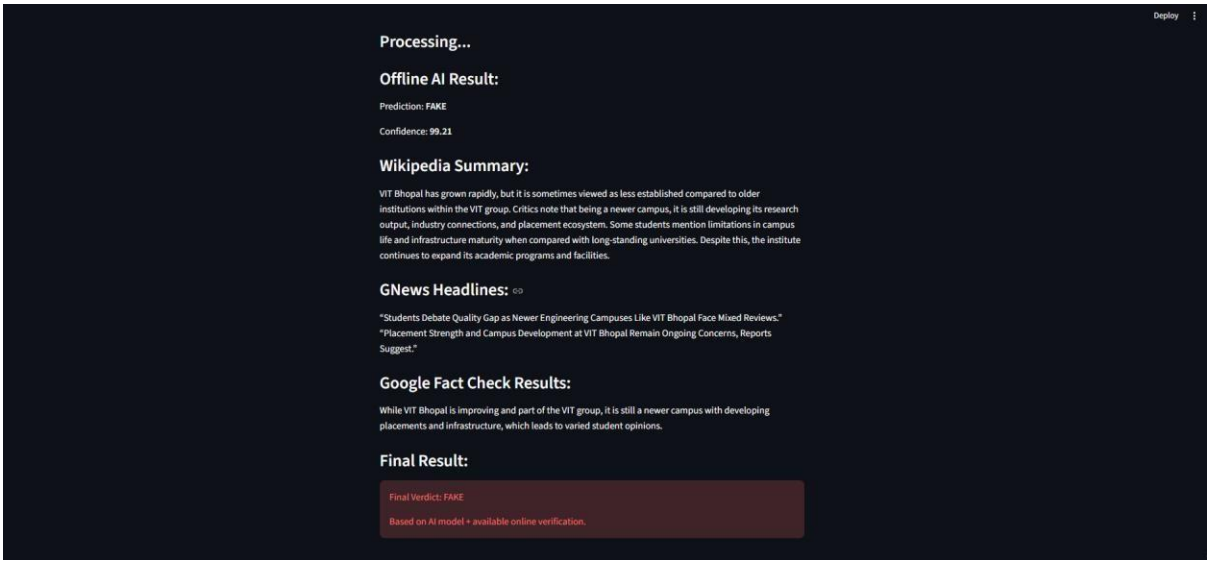
5. Final output is shown to the user

```
(venv) PS C:\FakeNews-main> streamlit run streamlit_app.py
>>

      Welcome to Streamlit!

      If you'd like to receive helpful onboarding emails, news, offers, promotions,
      and the occasional swag, please enter your email address below. Otherwise,
      leave this field blank.

      Email: abc@email,com

  You can find our privacy policy at https://streamlit.io/privacy-policy

  Summary:
  - This open source library collects usage statistics.
  - We cannot see and do not store information contained inside Streamlit apps,
    such as text, charts, images, etc.
  - Telemetry data is stored in servers in the United States.
  - If you'd like to opt out, add the following to %userprofile%/.streamlit/config.toml,
    creating that file if necessary:

    [browser]
    gatherUsageStats = false


  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://10.152.210.145:8501
```

8. Streamlit User Interface

I developed a candid Streamlit-based interface to allow the users to interact with the system. Through this interface, users can:

- Enter any news text for analysis
- Select between AI-only prediction and hybrid prediction modes
- View supporting information, including a Wikipedia summary, GNews results, and Fact-Check responses



9. Results and Discussion

The model performed extremely well in offline testing, with an accuracy close to 99 percent.

The hybrid system performed even better in real-world cases as it doesn't only rely on the model's prediction.

One of the main advantages of this approach is that it reduces the chance of misclassification. Even if the model is uncertain, the online fact-checking stage helps in the correction of the decision.

However, the hybrid system depends on the internet and free API limits, which can be a limitation when running many queries.

10. Conclusion

This project shows how a hybrid approach can make fake news detection more effective. By combining a deep learning model with online factchecking, the system becomes more reliable and better suited for real-world use.

The Streamlit interface also makes it easy for users to try it out and test different news articles without needing any technical background.

Overall, the project was able to achieve its goals and performed well both during offline evaluation and in real-world testing.

11. Future Improvements

If I continue working on this project, I would like to:

- Use a stronger model like DeBERTa or RoBERTa
- Add more APIs for verification
- Improve Wikipedia matching using semantic search
- Deploy the system online using a cloud platform
- Extend support to image-based or video fake news

12. References

- WELFake Dataset
- HuggingFace Transformers Library
- PyTorch Documentation
- Wikipedia API
- Google Fact Check Tools API
- GNews API